Performance test report

lendly · http://localhost:4173/lendly · 2026-10-22

1. Results at a glance

Test	Requests	Req/s	Avg ms	p90 ms	p95 ms	p99 ms	Max ms	Errors	Peak VUs	Verdict
smoke	30	1.0	1.9	4.0	4.3	4.4	4.5	0.00%	1	**PASS**
load	7520	41.6	0.5	0.8	1.1	2.1	4.4	0.00%	50	**PASS**

Stress and spike tests deliberately exceed normal load; they find limits and are not pass/fail gated.

2. SLO checks

smoke

Metric	Threshold	Result
http_req_failed	`rate<0.01`	**PASS**
http_req_duration{endpoint:items}	`p(95)<100`	**PASS**
http_req_duration{endpoint:home}	`p(95)<100`	**PASS**
http_req_duration{endpoint:bundle}	`p(95)<250`	**PASS**
http_req_duration{endpoint:people}	`p(95)<100`	**PASS**
http_req_duration	`p(95)<100`	**PASS**
http_req_duration	`p(99)<250`	**PASS**

load

Metric	Threshold	Result
http_req_duration{endpoint:bundle}	`p(95)<250`	**PASS**
http_req_duration{endpoint:people}	`p(95)<100`	**PASS**
http_req_failed	`rate<0.01`	**PASS**
http_req_duration{endpoint:items}	`p(95)<100`	**PASS**
http_req_duration{endpoint:home}	`p(95)<100`	**PASS**
http_req_duration	`p(99)<250`	**PASS**
http_req_duration	`p(95)<100`	**PASS**

3. Per-endpoint latency

smoke

Endpoint	Avg ms	Median ms	p95 ms	p99 ms	Max ms
home	1.2	1.1	1.7	1.8	1.8
items	1.3	1.2	1.8	1.9	2.0
people	1.0	1.0	1.0	1.0	1.0
bundle	3.1	3.8	4.4	4.5	4.5

load

Endpoint	Avg ms	Median ms	p95 ms	p99 ms	Max ms
home	0.4	0.3	0.8	1.2	1.8
items	0.4	0.3	0.8	1.1	1.9
people	0.4	0.3	0.8	1.1	1.6
bundle	0.7	0.6	1.8	2.5	4.4

4. Limits under stress and spike

Not run.

4b. Browser measurements

Cold load (empty cache; median)

Variant	FCP	LCP	TBT	CLS	Requests	Transfer	LCP < 2.5 s, TBT < 200 ms, CLS < 0.1
desktop, ads blocked	40 ms	40 ms	0 ms	0	4	313 KB	**PASS**
phone (4x CPU, slow 4G), ads blocked	450 ms	450 ms	73.5 ms	0	4	313 KB	**PASS**
phone (4x CPU, slow 4G), with ads	452 ms	452 ms	76 ms	0	13	313 KB	**PASS**

Data scale (localStorage seeded with N loans, 30% active)

Profile	Loans	Startup	Tab switch	Add loan	Mark returned	Heap	DOM nodes	Add loan < 100 ms
desktop	0	76.4 ms	39.1 ms	10.9 ms	8 ms	7.3 MB	70	**PASS**
desktop	100	104.1 ms	81.5 ms	26.2 ms	20.5 ms	9.7 MB	309	**PASS**
desktop	1,000	115.8 ms	280.4 ms	247.1 ms	237.1 ms	19.7 MB	2,469	**FAIL**

desktop	2,500	148.4 ms	1.43 s	1.96 s	1.93 s	39.4 MB	6,069	**FAIL**
desktop	5,000	209.3 ms	7.86 s	12.07 s	11.92 s	65 MB	12,069	**FAIL**
desktop	20,000	-	-	-	-	-	-	**DID NOT COMPLETE**
phone-4x	0	301.7 ms	80 ms	19.8 ms	9.5 ms	6.2 MB	65	**PASS**
phone-4x	100	425.5 ms	137.4 ms	62.1 ms	45.2 ms	9.5 MB	309	**PASS**
phone-4x	1,000	585.8 ms	1.11 s	1.16 s	1.06 s	19.6 MB	2,469	**FAIL**
phone-4x	2,500	647.9 ms	5.91 s	8.30 s	8.16 s	37.8 MB	6,069	**FAIL**
phone-4x	5,000	843.4 ms	32.65 s	51.10 s	52.01 s	68.6 MB	12,069	**FAIL**
phone-4x	20,000	-	-	-	-	-	-	**DID NOT COMPLETE**

Indicative: shared laptop CPU, CPU throttling approximates a phone, dataset sizes are assumptions.

Storage limit

localStorage accepted about 5.0 M characters; a write near the limit persisted.

5. Analysis and recommendations

Written by the analyst agent from the numbers above (bottlenecks, likely causes, fixes, re-test plan). See reports/analysis.md.

Analysis: lendly performance

Verdict
- **Delivery (k6, local copy of the static site): PASS.** Smoke and load (50 virtual users, 2 min hold) met every threshold: p95 about 4 ms against a 100 ms SLO, p99 about 6 ms, 0% errors, about 42 req/s. There is no server in the request path, so this only shows the static files are cheap to serve. It does not predict GitHub Pages behaviour.
- **First load (Playwright): PASS.** Phone profile (4x CPU, slow 4G): LCP 0.45 s against 2.5 s, TBT 74 ms against 200 ms, CLS 0. One 313 KB gzipped bundle, 4 requests.
- **Interactions at scale: FAIL beyond about 1,000 loans.** Adding a loan or marking one returned stays under 100 ms up to about 100 loans, then degrades faster than the data grows.

Dataset-size curve (median of 3 to 10 runs, 30% of loans active)
Loans	Desktop add loan	Phone (4x CPU) add loan	Desktop tab switch	Phone tab switch	DOM nodes
0	11 ms	20 ms	39 ms	80 ms	70
100	26 ms	62 ms	82 ms	137 ms	309
1,000	247 ms	1.2 s	280 ms	1.1 s	2,469

2,500	2.0 s	8.3 s	1.4 s	5.9 s	6,069
5,000	12.1 s	51.1 s	7.9 s	32.7 s	12,069
20,000	did not complete in 90 s	did not complete in 90 s	-	-	-

Going from 1,000 to 5,000 loans (5x) makes add-loan about 49x slower on desktop and 44x on phone: close to quadratic. Startup grows gently (76 ms to 209 ms desktop, 0 to 5,000 loans), so the cost is in updates, not loading.

Caveats: dataset sizes are assumptions (no usage data was available), 5,000 and 20,000 loans are far above what a personal lending tracker is likely to hold, 4x CPU throttling approximates but is not a real phone, and the laptop was shared with the test tooling. Use these as relative numbers.

Top bottlenecks
1. **Every mutation blocks the main thread for almost the whole interaction.** Long-task time is 243 ms of the 247 ms add-loan at 1,000 loans, and 12,046 of 12,067 ms at 5,000. The UI is frozen, not waiting on I/O.
2. **Work per update grows faster than linearly.** The strategy review flagged `src/lib/store.ts` (persist rewrites every key on each change; per-item loan lookups) and unvirtualised lists on Home, Items and People. The DOM reaching 12,069 nodes at 5,000 loans is consistent with rendering every row. These are likely causes, not profiled ones: confirm with a CPU profile before changing code.
3. **Memory grows with data**: 65 MB heap at 5,000 loans versus 7 MB empty. Not a problem on its own.
4. **Storage ceiling**: localStorage accepted about 5.2 million characters. The seeded 5,000-loan dataset uses about 1.1 MB, so quota is not the first limit; speed is. A write near the quota persisted without a visible error in this probe, but the app logs no warning if a real write ever fails.

Ranked fixes (not applied; the app source was not changed)
1. Make the list screens virtualised (FlatList-style windowing or pagination), so DOM size stops growing with data.
2. Persist only the changed collection and debounce writes, instead of rewriting all keys per mutation.
3. Index loans by item and person (maps built once) instead of scanning on every row.
4. Archive or collapse returned loans on Home (only active loans are needed there).
5. Handle storage write failures visibly.

Re-test plan
Re-run `npm run test:browser` (data-scale and cold-load) after each fix and compare against `results/browser-data-scale.json` from this run. Target: add-loan and mark-returned under 100 ms at 1,000 loans on the phone profile. Then `npm run test:load` and `npm run compare` to confirm delivery numbers did not regress. A 30-minute soak was not run; for a client-only app it is only relevant to memory growth in a long session.

Not covered
- Real devices, native iOS and Android, the live GitHub Pages site (never load-tested), and the AdSense script cost beyond one "with ads" cold-load variant (adds 9 requests and about 8 MB heap, +0.14 s script time; LCP unchanged).

6. Test setup

Tool: k6. Think time 1s. Target VUs 50. Default SLO: p95 < 100 ms, p99 < 250 ms, error rate < 1%. Raw data: `results/*.json`.